Counterfactual Explanations for Multivariate Time Series Forecasting

A Unified Framework and Empirical Evaluation

Annex Report

Detailed Results, Figures, and Program Listings

Kadidiatou Diallo

ENSEA, ETIS - Cergy, France

Supervisor : Vassilis Christophides

Master Research Project - 2024/2025

GitHub repository:

https://github.com/kadidiatou131313/forecastcf-univariate-ts-counterfactuals

Contents

1 Overview **2**

2 E0 - Forecasting Benchmark **2**

3 E1 - Forecasting Performance vs. CF Quality **3**

4 E2 - Comparison with Naive Baselines **5**

5 E3 - Sensitivity to Target Tolerance Width **8**

6 Program Listings **10**

6.1 `a2_forecastcf.py` - ForecastCF (PyTorch port) 11

6.2 `a2bis_forecastcf.py` - Baseline Methods 11

6.3 `a3Metrics.py` - Counterfactual Evaluation Metrics 11

6.4 `forecast_e0.py` - Forecasting Model Wrappers 11

7 Key Implementation Notes **12**

7.1 PyTorch port of ForecastCF 12

7.2 Adaptive target bound formulation 12

7.3 Random window sampling in E1 and E2 12

7.4 NNCF implementation detail 12

8 Figure Reference Summary **13**

1 Overview

This annex accompanies the research article *Counterfactual Explanations for Multivariate Time Series Forecasting: A Unified Framework and Empirical Evaluation*. It provides:

- the complete set of figures produced by the experimental notebook, organized by experiment;
- commented excerpts of the four Python modules developed for this project;
- implementation notes on the key design choices made when adapting the original ForecastCF code to a PyTorch/Darts environment.

All experiments were conducted on a standard laptop (CPU only, no GPU). Reproducibility is ensured by a fixed random seed (`SEED = 42`) and by the pre-trained model checkpoints provided in the github.

2 E0 - Forecasting Benchmark

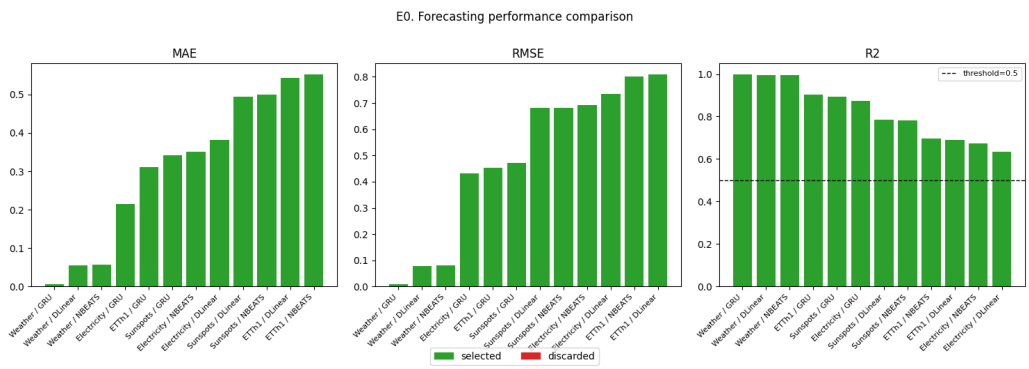
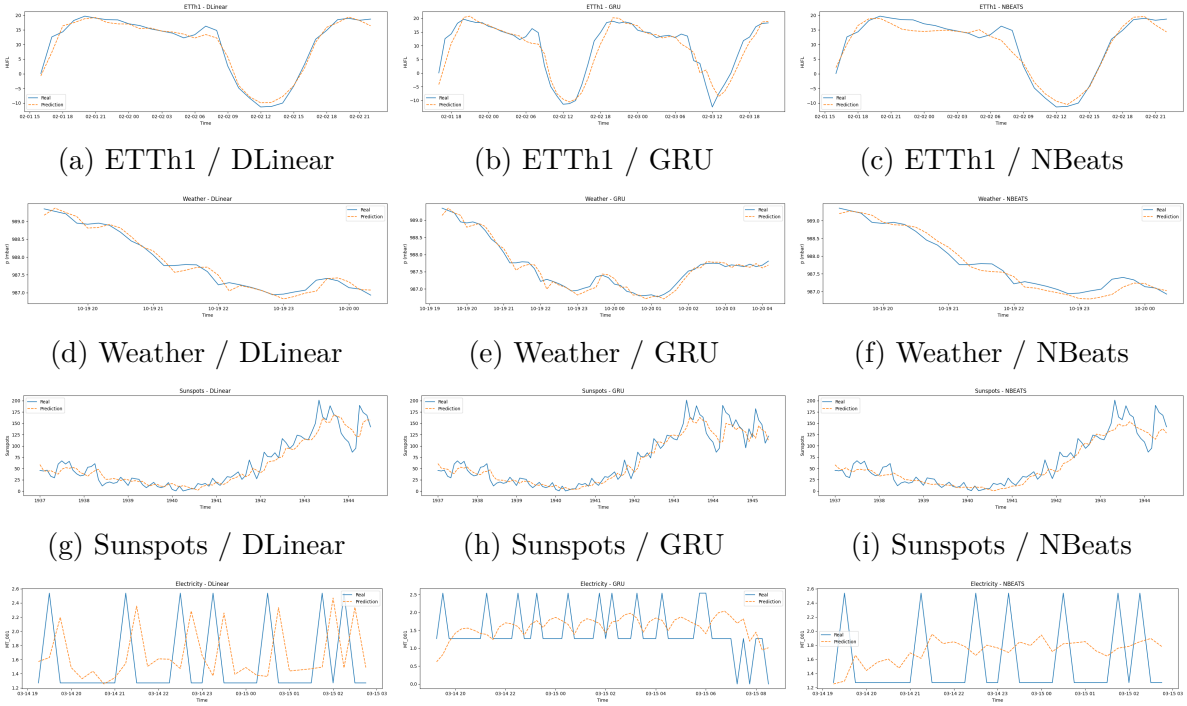


Figure 1: R^2 values for all 12 (dataset, model) pairs, sorted in descending order and colored by architecture. Dashed line : $R^2 = 0.50$ retention threshold. All 12 pairs are retained (minimum : 0.636, Electricity/DLinear; maximum : 0.9999, Weather/GRU).



(a) ETTh1 / DLinear (b) ETTh1 / GRU (c) ETTh1 / NBeats

(d) Weather / DLinear (e) Weather / GRU (f) Weather / NBeats

(g) Sunspots / DLinear (h) Sunspots / GRU (i) Sunspots / NBeats

(j) Electricity / DLinear (k) Electricity / GRU (l) Electricity / NBeats

Figure 2: Rolling-origin forecast visualizations for all 12 pairs. Blue : ground truth. Orange dashed : model prediction.

3 E1 - Forecasting Performance vs. CF Quality

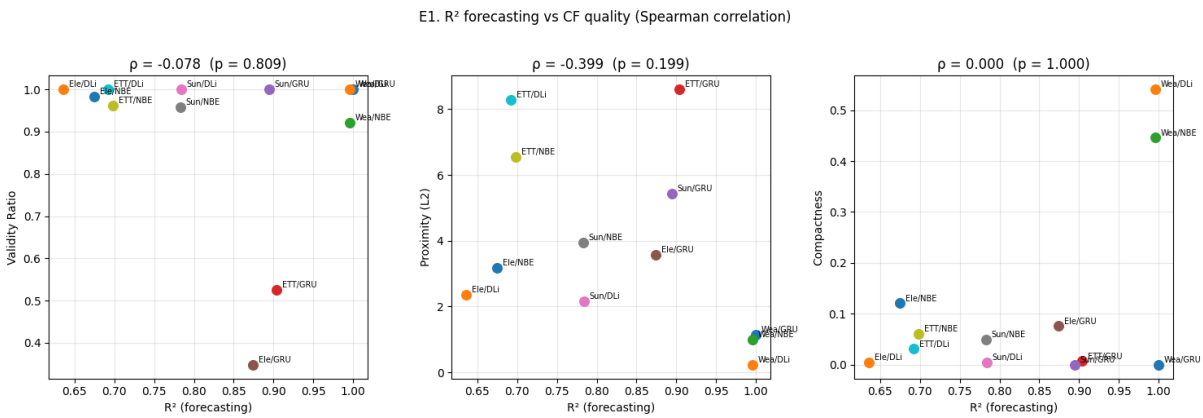


Figure 3: Scatter plots of R^2 vs. each CF metric across all 12 pairs. No significant Spearman correlation is found (Validity : ρ=−0.078, p=0.81; Proximity : ρ=−0.399, p=0.20; Compactness : ρ=0.000, p=1.00).

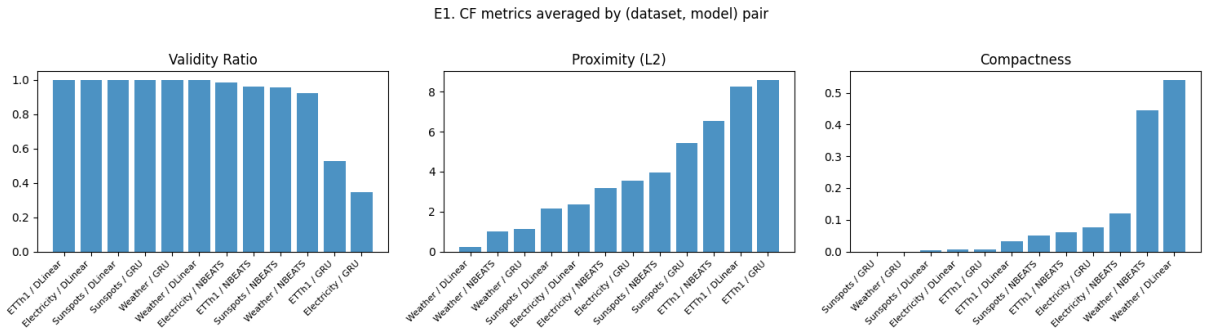


Figure 4: Grouped bar charts of Validity, Proximity, and Compactness per (dataset, model) pair. Error bars : ± 1 std over 10 instances. Compactness is near zero for almost all pairs, reflecting the absence of a sparsity term in ForecastCF's loss function.

4 E2 - Comparison with Naive Baselines

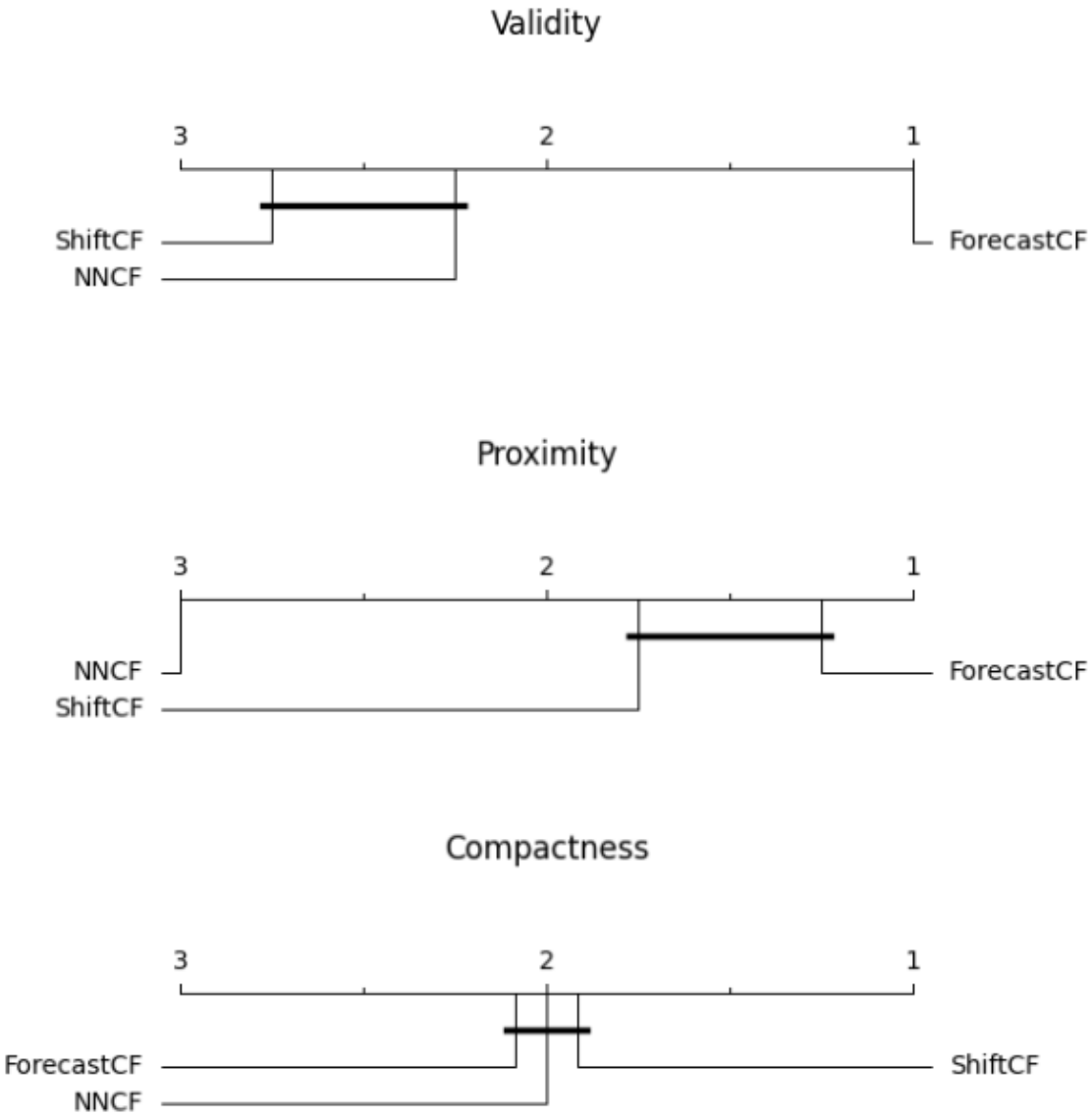


Figure 5: Critical Difference (CD) diagrams for Validity, Proximity, and Compactness across the 12 pairs (k=3, Nemenyi α=0.05, CD = 0.957). Axis : worst rank on the left, best rank on the right. Bold bar : no significant difference between connected methods. ForecastCF ranks first on Validity (avg. 1.00) and Proximity (1.25); all three methods are within CD on Compactness.

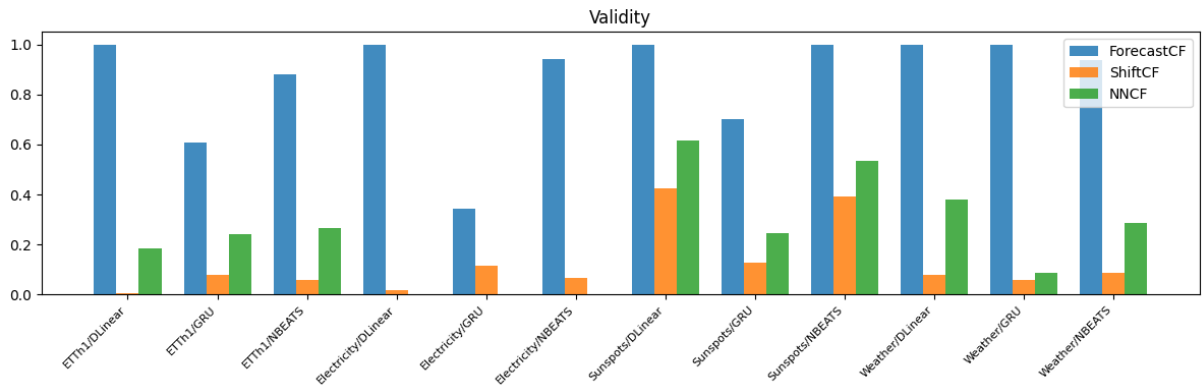


Figure 6: Mean Validity per (dataset, model) pair for the three methods. ForecastCF achieves the highest validity on all pairs.

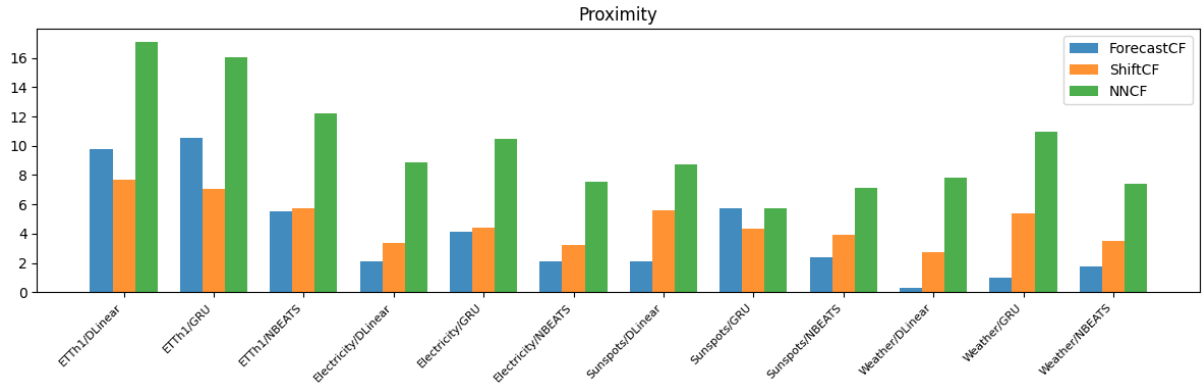


Figure 7: Mean Proximity per pair. ForecastCF is significantly closer to the original than NNCF (p<0.0001); the smaller advantage over ShiftCF (p=0.0085) reflects that a uniform multiplicative shift is itself compact, but has no validity guarantee.

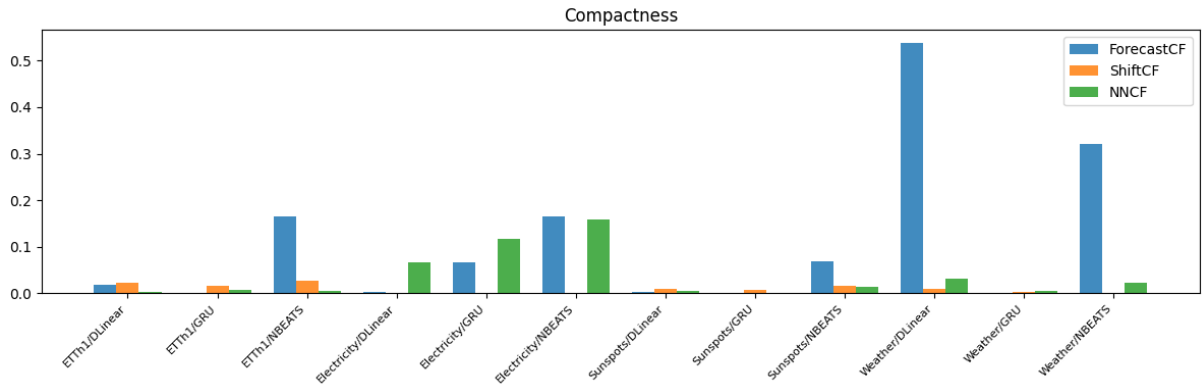


Figure 8: Mean Compactness per pair. Near-zero scores across all methods confirm that perturbation-based approaches without explicit sparsity regularization modify nearly every input timestep.

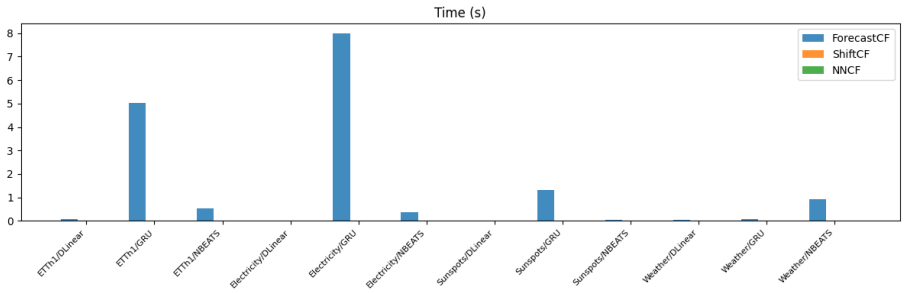


Figure 9: Generation time per instance (seconds, log scale). ForecastCF ranges from < 0.01s (DLinear) to $\sim$8s (GRU/Electricity). ShiftCF and NNCF are near-instantaneous.

5 E3 - Sensitivity to Target Tolerance Width

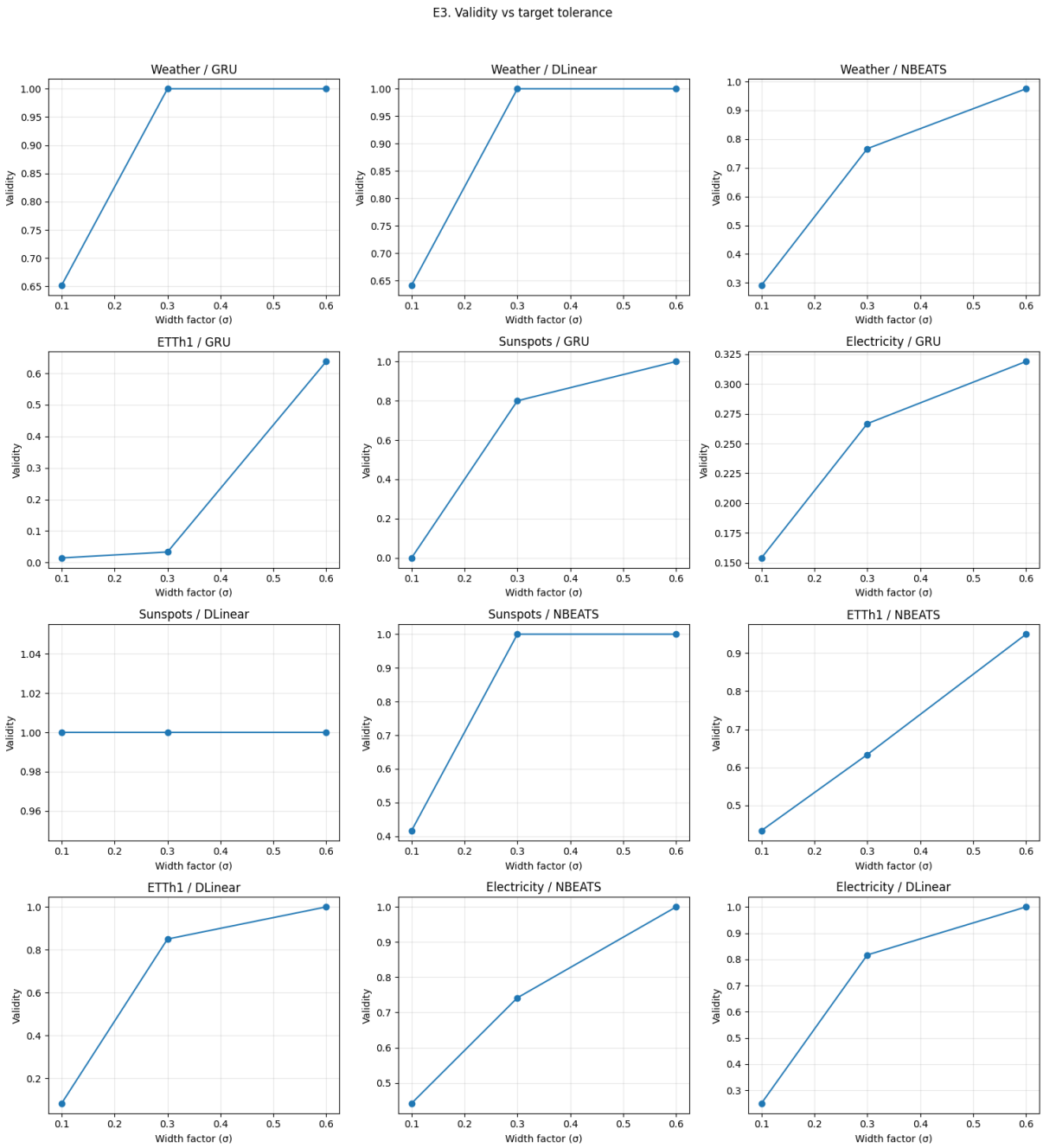


Figure 10: Validity vs. tolerance width w for all 12 pairs. Validity increases with w for almost all pairs. ETTh1/GRU and Electricity/GRU plateau below 0.65 even at 0.6σ, revealing the structural difficulty of tight-constrained recurrent models.

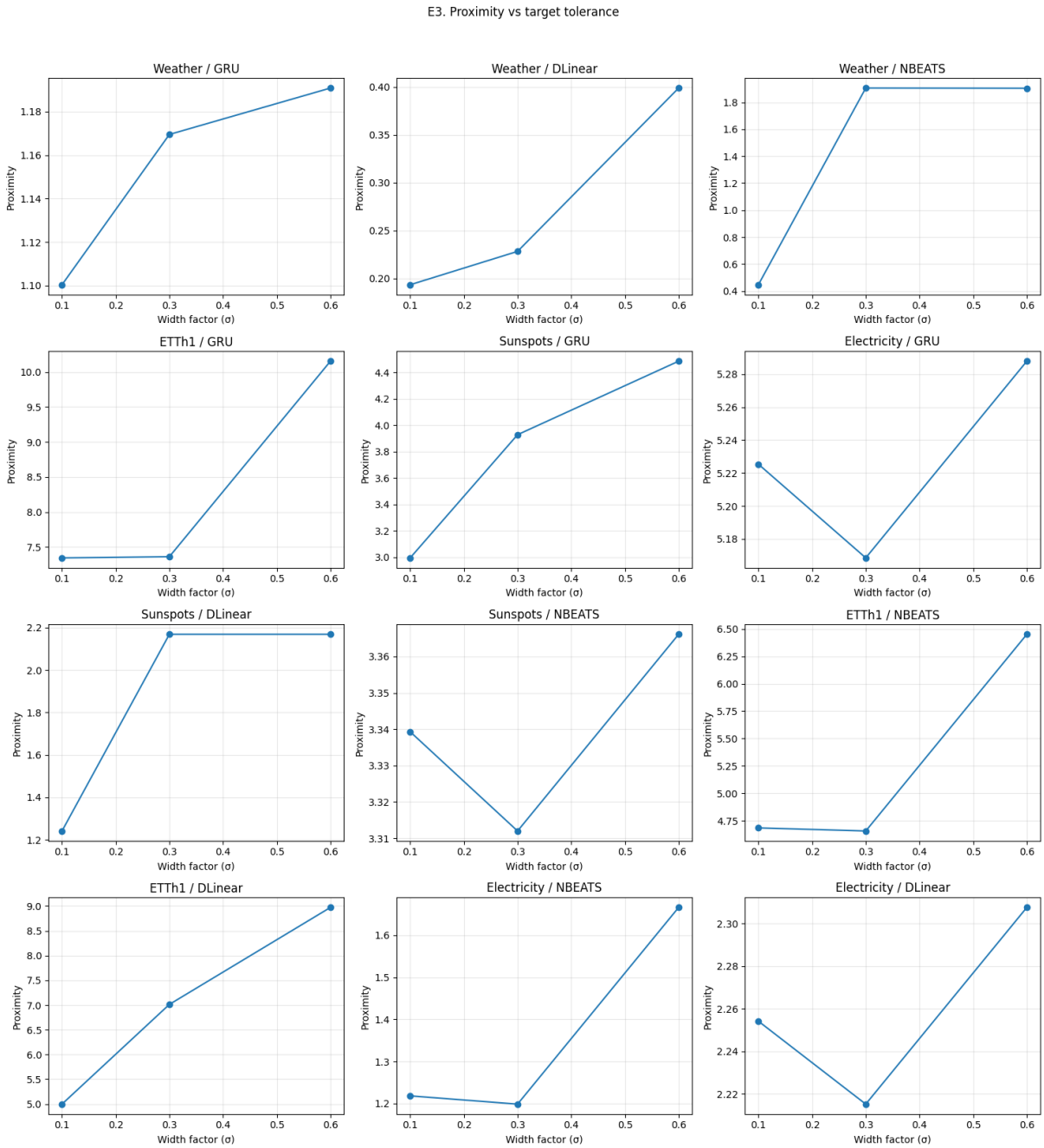


Figure 11: Proximity vs. tolerance width w for all 12 pairs. Proximity generally increases with w : a wider band placed beyond the original prediction requires larger input perturbations. Non-monotonic exceptions at 0.3σ are attributed to the small sample size (N=5).

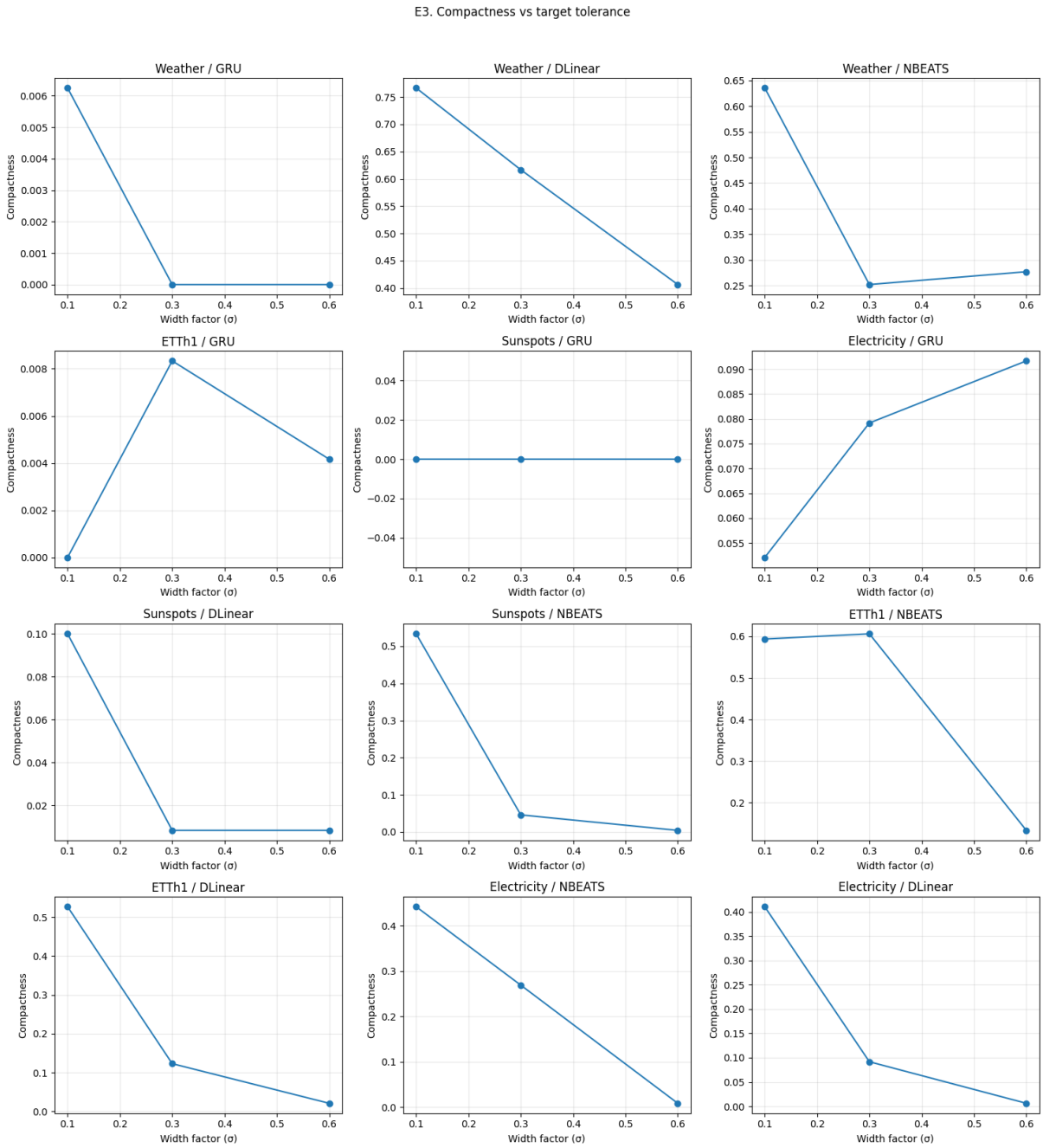


Figure 12: Compactness vs. tolerance width w for all 12 pairs. Compactness decreases with w, confirming the validity-compactness trade-off : achieving a wider target requires modifying more input timesteps.

6 Program Listings

This section provides commented listings of the four Python modules developed for this project. The complete codes are available in github.

6.1 `a2_forecastcf.py` - ForecastCF (PyTorch port)

The original ForecastCF implementation by Wang et al. (2023) was built in TensorFlow/Keras. This module is a full PyTorch re-implementation that integrates directly with the Darts-based forecasting models used in these experiments (which expose internal PyTorch modules via `model.model`).

The core logic follows Algorithm 1 of the paper : iterative Adam updates on the input series until all forecast timesteps fall within the target bounds, controlled by a masking-based margin loss. A key observation about this implementation : the loss function contains *no proximity term*, meaning ForecastCF optimizes validity exclusively. This is a deliberate design choice from the original paper, and a central point of analysis in Section III-B of the main article.

6.2 `a2bis_forecastcf.py` - Baseline Methods

This module extends `a2_forecastcf.py` with two naive baseline counterfactual generators used in E2.

BaselineShiftCF applies a fixed multiplicative shift to the input series and serves as the simplest possible perturbation strategy. It requires no model access.

BaselineNNCF retrieves the nearest neighbor from the training set whose corresponding model prediction is closest (in ℓ_2 distance) to the center of the target interval. It requires a prior call to `.fit(X_train, Y_train)` to build the neighbor index using `sklearn.neighbors.NearestNeighbors`.

6.3 `a3Metrics.py` - Counterfactual Evaluation Metrics

This module implements the four counterfactual evaluation metrics defined in Wang et al. (2023), adapted from their original NumPy/Pandas formulation. One notable adaptation : `stepwise_validity_auc` uses `np.trapezoid` instead of the deprecated `np.trapz` available in recent NumPy versions (≥ 2.0).

6.4 `forecast_e0.py` - Forecasting Model Wrappers

This module provides the wrapper classes that bridge the Darts model API with the experimental pipeline. The key entry point is `build_base_configuration`, which returns a constructor and a configuration dictionary for each supported architecture.

7 Key Implementation Notes

7.1 PyTorch port of ForecastCF

The original ForecastCF code uses TensorFlow/Keras. The main differences in this PyTorch port are:

- The input tensor is created with `requires_grad=True` and passed directly to `torch.optim.Adam`, which handles the gradient accumulation and parameter update internally.
- The model is called as `torch_model((x, None, None))` to match the input signature expected by Darts' internal PyTorch modules, which take a tuple `(x_in, x_mark, x_dec)`.
- The `dtype` parameter is dynamically inferred from the model parameters (`next(model.parameters()).dtype`) to avoid float32/float64 mismatches between the model and the optimizer.

7.2 Adaptive target bound formulation

Unlike the original paper which uses a fixed fraction of the training set standard deviation, this implementation normalizes bounds by the standard deviation of each individual input window (σ_x). This design choice avoids the need for any dataset-level calibration and ensures that the target constraint is always meaningful regardless of the local signal amplitude.

7.3 Random window sampling in E1 and E2

The original ForecastCF paper evaluates CF quality on the full test set. In this work, 10 windows are sampled uniformly at random from the test set (without replacement) for each pair. This avoids temporal position bias while keeping the per-pair evaluation tractable. The random seed is fixed (`SEED = 42`) for reproducibility.

7.4 NNCF implementation detail

For computational efficiency, the NNCF training index is built on a subsample of at most 1000 training windows (stride 20). This covers the full temporal range of the training set while limiting memory usage. The `sklearn.neighbors.NearestNeighbors` index uses Euclidean distance on the flattened prediction vectors (`Y_train`).

8 Figure Reference Summary

Table 1: Summary of all figures produced by the notebook, with their file names and the section of this annex where they appear.

File name	Description	Section
`E0_r2_per_fold.png`	R^2 bar chart, all 12 pairs	1
`E0_{Dataset}_{Model}.png`	Forecast plots (12 files)	2
`e1_scatter.png`	R^2 vs. CF metrics scatter	3
`E1_CF_metrics_per_pair.png`	CF metrics grouped bars	4
`fig_e2_ranking.png`	CD diagrams Validity / Proximity	5
`E2_Validity.png`	Validity per pair, 3 methods	6
`E2_Proximity.png`	Proximity per pair, 3 methods	7
`E2_Compactness.png`	Compactness per pair, 3 methods	8
`E2_Time_s_.png`	Generation time per pair	9
`E3_Validity.png`	Validity vs. w, 12-pair grid	10
`E3_Proximity.png`	Proximity vs. w, 12-pair grid	11
`E3_Compactness.png`	Compactness vs. w, 12-pair grid	12